

Programmation et Algorithmique I

Test du 10 novembre 2015

Consignes :

- répondez à chaque problème **sur des feuilles séparées** et utilisez uniquement les feuilles de réponses qui vous sont données ;
 - indiquez votre nom, prénom et section sur chaque feuille de réponses ;
 - les notes de cours et personnelles **ne sont pas autorisées** ;
 - si ce n'est pas explicitement demandé dans l'énoncé :
 - **vous ne devez pas tester que les préconditions sont respectées** ;
 - **vos fonctions ne doivent pas avoir d'effet de bord** : les objets mutables passés en paramètres ne doivent pas être modifiés et rien ne doit être affiché « à l'écran ».
-

Problème 1 : factorisation en nombres premiers

Pour rappel, un nombre entier $n \geq 2$ est premier si et seulement s'il n'admet comme diviseurs que 1 et lui-même. La décomposition en facteurs premiers d'un nombre x est un ensemble de nombres premiers tel que le produit de ces nombres est égal à x . Étant donné un nombre entier $x \geq 2$, cette décomposition est unique. Considérez le code de la fonction `decomposition(x)` suivante qui permet de calculer la décomposition en facteurs premiers d'un nombre entier $x \geq 2$. Son principe est de diviser x par i tant que c'est possible, c'est à dire tant que x est divisible par i .

```
def decomposition(x):  
    i = 2  
    r = []  
    while x > 1:  
        while x % i == 0:  
            x = x / i  
            r.append(i)  
        i = i + 1  
    return r
```

Voici un exemple de résultats obtenus par la fonction :

```
>>> print decomposition(84)  
[2, 2, 3, 7]  
>>> decomposition(243)  
[3, 3, 3, 3, 3]  
>>> print decomposition(17)  
[17]
```

Questions :

- a) On suppose (précondition) que le nombre x donné en entrée est un entier $x \geq 2$. Pour chacune des assertions suivantes, indiquez s'il s'agit d'un invariant de la boucle **externe** (`while x > 1`). Si oui, prouvez qu'il est respecté, sinon, expliquez pourquoi ce n'est pas un invariant de boucle.
- (1) `r` contient tous les nombres premiers plus petits que `i`.
 - (2) `r` contient tous les diviseurs de x plus petits que `i`.

(3) x contient tous les diviseurs premiers de x qui sont strictement inférieurs à i .

- b) Etant donné un nombre x qui possède k facteurs premiers, quelle est la complexité, en notation grand- O , de la fonction `decomposition(x)` ? Justifiez votre réponse.

Problème 2 : insertion dichotomique

Le but de cette question est de créer un ensemble de fonctions permettant d'insérer un entier au bon endroit dans une liste déjà triée par ordre croissant. Par exemple supposons la liste :

```
listA=[2, 4, 5, 8, 9]
```

Après l'insertion du chiffre '7' la liste devient

```
[2, 4, 5, 7, 8, 9]
```

Avant d'insérer l'élément dans la liste il faudra donc d'abord déterminer en quelle position il doit être inséré. Pour ce faire le principe **récuratif** de la recherche **dichotomique** peut être utilisé.

Ce principe commence par deux entiers a et b représentant l'intervalle de recherche. Ces points sont choisis de telle sorte que la valeur k recherchée appartienne à l'intervalle $[a; b]$ (*précondition*). À chaque étape de l'algorithme, l'intervalle $[a; b]$ est découpé en deux intervalles de même longueur (à une unité près, car nous travaillons avec des entiers). Notons-les $[a; c]$ et $[c; b]$. Ensuite,

- si c est la valeur recherchée, alors l'algorithme retourne cette valeur ;
- sinon, on passe à l'étape suivante avec l'un des intervalles vérifiant la précondition précitée.

Questions :

- a) Donnez le code python d'une fonction **réursive** de recherche **dichotomique** `getIndex(myList, elem)` qui, étant donné une liste triée par ordre croissant `myList` et un entier à insérer `elem`, retourne l'indice correspondant à la position où `elem` doit être inséré. Par exemple l'appel suivant retournera '3' :

```
getIndex([2, 4, 5, 8, 9], 7)
```

Il est impératif que votre fonction fasse appel à une recherche dichotomique.

- b) Donnez le code python d'une fonction `insert(myList, indice, elem)` qui, étant donné une liste `myList`, viendra insérer l'entier `elem` en position `indice`. Voici un exemple d'exécution qui devrait afficher `[2, 4, 5, 7, 8, 9]` à l'écran :

```
myList=[2, 4, 5, 8, 9]
insert(myList, 3, 7)
print myList
```